

Memoria técnica incremental del proyecto (Hitos 1, 2 y 3)

Asignatura: Desarrollo y Despliegue de Soluciones Big Data

Máster: Máster Universitario en Big Data y Computación en la Nube

Curso académico: 2025–2026

Alonso Marcos Muñoz

Alonso.Marcos@alu.uclm.es

Jose Barros Ribademar

Jose.Barros1@alu.uclm.es

Abril de 2026

Índice

1. Hito 1: Alcance y viabilidad	3
1.1. Introducción del hito	3
1.2. Alcance y viabilidad	3
1.2.1. Definición del problema de negocio	3
1.2.2. Análisis y selección de la solución técnica	4
1.2.3. Evaluación de la viabilidad	4
1.3. Planificación y recursos	5
1.3.1. Objetivos de negocio y traducción técnica	5
1.3.2. Organización del equipo	5
1.3.3. Fuentes de datos y stack tecnológico	5
1.3.4. Cronograma oficial del proyecto	6
1.4. Cierre del Hito 1	6
2. Hito 2: Preparación y gestión de datos	6
2.1. Objetivo del hito	6
2.2. Implementación en Databricks	6
2.3. Fuentes de datos y ventana temporal	7
2.4. Arquitectura medallion ejecutada	7
2.4.1. Capa bronze: ingesta y trazabilidad	8
2.4.2. Capa silver: calidad, historial y entidades limpias	9
2.4.3. Capa gold: base de modelado y características	10

2.5.	Control de calidad – Capa Silver	11
2.5.1.	Centralización de reglas	11
2.5.2.	Reglas por entidad	11
2.6.	Robustez y resolución de incidencias	13
2.7.	Orquestación final del hito	14
2.8.	Decisiones de limpieza y gobierno del entorno	14
2.9.	Estado de cierre del Hito 2	15
3.	Hito 3: Modelado y experimentación	16
3.1.	Objetivo del hito	16
3.2.	Componentes implementados	16
3.3.	Generación del dataset de entrenamiento	16
3.4.	Arquitectura del pipeline de modelado	17
3.5.	Experimentación con MLflow	18
3.6.	Registro en Unity Catalog y ciclo champion-challenger	19
3.7.	Orquestación y evidencia de ejecución	19
3.8.	Cierre del Hito 3	20
4.	Hito 4: Despliegue y monitorización (apartado reservado)	20

Datos de la entrega

- Asignatura: **Desarrollo y Despliegue de Soluciones Big Data.**
- Máster: **Máster Universitario en Big Data y Computación en la Nube.**
- Curso académico: **2025–2026.**
- Integrantes:
 - Alonso Marcos Muñoz (Alonso.Marcos@alu.uclm.es).
 - Jose Barros Ribademar (Jose.Barros1@alu.uclm.es).

1. Hito 1: Alcance y viabilidad

1.1. Introducción del hito

El objetivo de este primer hito es cerrar el alcance del proyecto con una justificación técnica y económica suficiente para sustentar su ejecución durante el resto de la asignatura. En esta fase no se plantea un prototipo parcial, sino una propuesta completa de trabajo que permita defender qué problema de negocio se pretende resolver, por qué una arquitectura de datos distribuida es adecuada y qué condiciones de viabilidad deben cumplirse para que el proyecto sea defendible en términos académicos y profesionales.

La memoria se ha redactado como documento de base para los siguientes hitos, de forma que cada decisión tomada en febrero de 2026 quede trazada con su hipótesis asociada y pueda validarse posteriormente con resultados de implementación. La fecha oficial de cierre del hito es el **27 de febrero de 2026**.

1.2. Alcance y viabilidad

1.2.1. Definición del problema de negocio

La empresa de telecomunicaciones analizada gestiona una cartera activa de 2.000.000 de clientes particulares en un mercado muy competitivo. En el escenario de partida se observa una rotación mensual del 3%, equivalente a 60.000 bajas cada mes. Esta dinámica refleja que las campañas generalistas de retención no están siendo suficientes para priorizar a los clientes con mayor probabilidad de abandono y mayor impacto económico.

Desde el punto de vista financiero, el problema se concentra en dos frentes. Por un lado, se estima que 40.000 bajas mensuales son potencialmente recuperables y que cada cliente perdido supone, de media, 100 euros de margen neto anual no capturado. Por otro lado, la compañía aplica descuentos de forma poco precisa sobre una parte de clientes que no presenta riesgo real de fuga, lo que incrementa el coste comercial sin retorno proporcional.

$$\text{Pérdida por fugas recuperables} = 40,000 \times 100 = 4,000,000 \text{ EUR/mes}$$

$$\text{Coste por incentivos mal asignados} = 100,000 \times 10 = 1,000,000 \text{ EUR/mes}$$

Impacto económico mensual estimado = 5,000,000 EUR/mes

Este volumen de pérdida justifica abordar el problema con una solución analítica que permita identificar mejor el riesgo de churn y mejorar la asignación de incentivos.

1.2.2. Análisis y selección de la solución técnica

Para resolver el caso se valoraron tres enfoques: reglas heurísticas, aprendizaje automático en entorno monolítico y arquitectura big data distribuida. El enfoque heurístico ofrece rapidez inicial, pero su mantenimiento crece de forma no lineal al aumentar la casuística y sufre cuando cambian los patrones de comportamiento. El enfoque monolítico con machine learning mejora la capacidad predictiva, aunque introduce limitaciones de escalabilidad, de trazabilidad operativa y de incorporación de nuevas fuentes en escenarios de mayor volumen.

La alternativa distribuida, basada en un pipeline de datos en Databricks con arquitectura medallion, se selecciona porque se ajusta mejor al tamaño de la base de clientes, permite evolucionar el sistema por capas (bronze, silver y gold) y facilita la gobernanza técnica del proyecto. Esta decisión no se fundamenta en complejidad tecnológica por sí misma, sino en su adecuación al problema y en su coherencia con los objetivos formativos de la asignatura.

1.2.3. Evaluación de la viabilidad

Viabilidad técnica La viabilidad técnica se considera favorable por cuatro motivos. En primer lugar, existe masa crítica de datos para entrenar y evaluar modelos de clasificación supervisada. En segundo lugar, el problema tiene una definición clara de variable objetivo (baja/no baja) y un marco temporal razonable para construir etiquetas y ventanas de observación. En tercer lugar, se dispone de señales potencialmente explicativas, como antigüedad del cliente, uso de servicios, incidencias, facturación e historial de interacción comercial. En cuarto lugar, la ejecución en modo batch reduce el riesgo de implementación temprana y permite priorizar robustez, reproducibilidad y calidad del dato antes de plantear escenarios de inferencia más exigentes.

Viabilidad económica El escenario económico se ha planteado con hipótesis conservadoras para evitar sobreestimar el impacto. Si el sistema consigue reducir un 5 % las fugas recuperables y, en paralelo, disminuir un 5 % el coste de incentivos mal dirigidos, el beneficio mensual esperado sería de 250.000 euros.

Beneficio por mejora en fugas = $4,000,000 \times 0,05 = 200,000$ EUR/mes

Beneficio por mejora en incentivos = $1,000,000 \times 0,05 = 50,000$ EUR/mes

Beneficio total estimado = 250,000 EUR/mes

En cuanto a costes, se considera una carga mensual de 10.000 euros de dedicación técnica y 1.100 euros de infraestructura cloud, lo que sitúa el coste total en 11.100 euros al mes.

$$\text{ROI mensual} = \left(\frac{250,000 - 11,100}{11,100} \right) \times 100 = 2,152 \%$$

$$\text{Payback aproximado} = \frac{11,100}{250,000} \times 30 = 1,3 \text{ días}$$

Aunque estos resultados dependen de hipótesis que deberán contrastarse en los hitos de datos y modelado, el orden de magnitud obtenido muestra que el caso tiene sentido económico incluso en un escenario prudente.

Viabilidad ética y legal La propuesta incorpora desde el inicio restricciones de cumplimiento alineadas con RGPD y con buenas prácticas de IA aplicada. Se prevéseudonimización de identificadores, control de accesos sobre datasets sensibles y trazabilidad de transformaciones y versiones de modelo. Además, se incluye evaluación de sesgo mediante métricas de equidad, con el objetivo de detectar desviaciones de rendimiento entre grupos y evitar decisiones comerciales sistemáticamente desfavorables para colectivos concretos.

1.3. Planificación y recursos

1.3.1. Objetivos de negocio y traducción técnica

Los objetivos operativos para el proyecto son reducir en un 5 % la pérdida asociada a fugas recuperables y reducir en un 5 % el coste de incentivos innecesarios. Traducido a magnitudes de seguimiento mensual, esto implica pasar de 40.000 a 38.000 bajas recuperables y de 100.000 a 95.000 incentivos potencialmente mal asignados. En términos técnicos, estos objetivos se transforman en tres líneas de trabajo: mejorar la capacidad de detección temprana del riesgo de churn, aumentar la precisión de segmentación para campañas de retención y mantener estabilidad de pipeline para garantizar repetibilidad de resultados.

1.3.2. Organización del equipo

El trabajo se organiza en una estructura colaborativa de dos perfiles complementarios. Una parte se centra en ingeniería de datos y MLOps, abarcando ingesta, control de calidad, orquestación y trazabilidad del pipeline. La otra parte se orienta al modelado, incluyendo construcción de variables, entrenamiento, evaluación y análisis de error. Esta distribución no es rígida; se revisa de forma continua para equilibrar carga y acelerar resolución de incidencias.

1.3.3. Fuentes de datos y stack tecnológico

La solución se apoyará en cuatro fuentes funcionales: maestro de clientes, histórico de bajas, registros de uso y facturación, e histórico de campañas con respuesta observada. Sobre estas fuentes se implementará un pipeline en Databricks y Delta Lake, con procesamiento distribuido en Spark, modelado con Spark MLlib y seguimiento experimental mediante

MLflow. El control de versiones y la coordinación del desarrollo se mantienen en GitHub para preservar trazabilidad técnica y facilitar auditoría del trabajo.

1.3.4. Cronograma oficial del proyecto

El calendario de la asignatura queda estructurado en cuatro hitos consecutivos: Hito 1 (Alcance y viabilidad) del 24/02/2026 al 27/02/2026; Hito 2 (Preparación y gestión de datos) del 28/02/2026 al 20/03/2026; Hito 3 (Modelado y experimentación) del 21/03/2026 al 17/04/2026; y Hito 4 (Despliegue y monitorización) del 18/04/2026 al 01/05/2026. La planificación del presente documento está alineada con ese marco oficial.

1.4. Cierre del Hito 1

Con este hito queda definido un alcance técnicamente consistente y económicamente justificado para el proyecto de churn telco. La propuesta establece el problema de negocio, argumenta la elección arquitectónica, delimita condiciones de viabilidad y fija objetivos medibles para los siguientes entregables. El siguiente paso es ejecutar el Hito 2 con foco en preparación del dato, validación de calidad y materialización del pipeline medallion en el entorno de trabajo.

2. Hito 2: Preparación y gestión de datos

2.1. Objetivo del hito

El objetivo real de este segundo hito ha sido pasar del diseño conceptual del Hito 1 a una implementación de datos que funcione de verdad en Databricks, con ejecución completa y repetible. En otras palabras, no se trataba solo de “tener scripts”, sino de validar que el proyecto podía desplegarse, correr sin intervención manual y dejar un resultado coherente para el inicio de Hito 3.

Para conseguirlo, el trabajo se organizó como un proceso incremental: primero establecimos el entorno de ejecución y el bundle, después cerramos la arquitectura medallion por capas, y finalmente incorporamos orquestación tipo producción (pipeline + notebook final), siguiendo el patrón del proyecto de referencia del profesor pero adaptado a nuestra temática de churn telco.

2.2. Implementación en Databricks

La implementación se articula en torno a Databricks Asset Bundles:

- `codigo/databricks.yml` como punto de entrada de configuración,
- `codigo/resources/telco_churn.pipeline.yml` para la definición del pipeline medallion,
- `codigo/resources/telco_churn.job.yml` para la orquestación final de Hito 2 en dos tareas,

- `codigo/resources/telco_churn_ml.job.yml` para la orquestación de modelado de Hito 3.

Una decisión importante de esta fase fue consolidar un único workspace operativo para evitar límites de cuota y problemas de permisos del entorno inicial. Sobre ese workspace se normalizó la capa de gobierno en Unity Catalog con una estructura explícita y descriptiva:

- Catálogo: `workspace`,
- Esquema operativo del proyecto: `workspace.telco_churn`,
- Volumen de entrada: `/Volumes/workspace/telco_churn/landing_zone`.

Además de la estructura, se añadieron descripciones visibles en catálogo, esquema, volumen y pipeline para que cualquier miembro del equipo pueda entender rápidamente qué recurso es oficial y cuál no, directamente desde la UI de Databricks, sin tener que abrir código local.

Durante la fase de despliegue apareció una incidencia crítica de operativa (`413 Request Entity Too Large`) al intentar sincronizar datos masivos junto con el bundle. La solución fue separar de forma estricta “código versionable” y “artefacto generado”, excluyendo del `sync` los directorios de datos (`context/**`, `events/**`, `source_buffer/**`) y logs de generación. Este ajuste fue clave para estabilizar `validate/deploy` y dejar un flujo reproducible para ambos integrantes del equipo.

2.3. Fuentes de datos y ventana temporal

La generación de datos sintéticos se realiza con `codigo/src/medallion_pipeline/utilities/generate.py`. Este script crea el maestro de clientes (`context/customers.csv`), los eventos mensuales de uso, interacciones y etiquetas en `events/YYYY/MM/data.json`, y una zona adicional `source_buffer` para datos de 2025 con deriva simulada de producción.

En la práctica, el script genera un volumen de información suficientemente grande para tensionar la ejecución en Free Edition y obligar a tomar decisiones técnicas reales de rendimiento. Esto era un objetivo del hito: demostrar un escenario cercano a Big Data, no un ejemplo mínimo.

Desde el punto de vista temporal, en Hito 2 se decidió mantener una frontera clara entre histórico y producción simulada:

- `events/*` se trata como histórico de entrenamiento (2023–2024),
- `source_buffer/*` se conserva como bloque de 2025 con deriva, útil para validación temporal posterior.

Esta separación evita mezclar periodos sin control y deja preparado el terreno para un Hito 3 metodológicamente correcto, especialmente en lo relativo a fuga temporal de información.

2.4. Arquitectura medallion ejecutada

A continuación se describe cada capa por separado para que quede claro qué responsabilidad tiene, qué tablas materializa y qué datos contiene cada salida. Los recuentos indicados corresponden a la carga histórica usada en Hito 2: `context/customers.csv` y

`events/*` para 2023–2024. Los ficheros de 2025 se mantienen en `source_buffer/*` como producción simulada con deriva y no entran automáticamente en estas tablas si no se amplía la ingesta.

2.4.1. Capa bronze: ingesta y trazabilidad

La capa bronze se implementa en `codigo/src/medallion_pipeline/transformations/01_bronze_ingestion.py`. Su responsabilidad es conservar el dato de entrada con la mínima transformación posible y añadir metadatos técnicos de auditoría. El maestro de clientes se lee como batch porque es una fotografía de contexto, mientras que uso, etiquetas e interacciones se ingieren con Auto Loader en modo incremental.

Tabla materializada	Filas	Contenido
<code>bronze_customers</code>	3.050.000	Maestro de clientes sintético. Mantiene atributos demográficos, contractuales, comerciales y de facturación, más trazabilidad de ingesta.
<code>bronze_usage</code>	32.121.558	Eventos mensuales de uso y facturación por cliente durante 2023–2024. Es la principal fuente de señales dinámicas para churn.
<code>bronze_labels</code>	32.121.558	Etiquetas mensuales de churn disponibles al cierre de cada mes. Permiten construir la variable objetivo sin mezclarla con el maestro de clientes.
<code>bronze_interactions</code>	17.021.772	Eventos de relación con el cliente: llamadas, chats, visitas, reclamaciones, soporte técnico y señales comerciales.

Columnas generadas en bronze:

- `bronze_customers`: `customer_id`, `customer_updated_at`, `age`, `gender`, `contract_type`, `region`, `region_type`, `tariff_plan`, `monthly_fee`, `num_lines`, `device_type`, `acquisition_channel`, `payment_method`, `signup_date`, `has_tv_bundle`, `has_fiber`, `has_roaming`, `paperless_billing`, `autopay`, `nps_score_at_start`, `is_active`, `ingestion_timestamp`, `source_file`.
- `bronze_usage`: `bill_amount`, `bill_overage`, `call_minutes`, `coverage_score`, `customer_id`, `data_consumed_gb`, `days_active`, `days_payment_late`, `nps_score`, `roaming_gb`, `sms_count`, `speed_mbps`, `tariff_plan`, `year_month`, `_rescued_data`, `ingestion_timestamp`, `source_file`.
- `bronze_labels`: `churn_date`, `customer_id`, `label_available_date`, `year_month`, `_rescued_data`, `ingestion_timestamp`, `source_file`.
- `bronze_interactions`: `agent_id`, `channel`, `customer_id`, `duration_seconds`, `interaction_type`, `resolution`, `satisfaction_score`, `timestamp`, `_rescued_data`, `ingestion_timestamp`, `source_file`.

La columna `_rescued_data` aparece en las tablas JSON gestionadas por Auto Loader y permite conservar campos inesperados sin romper la ejecución. En todas las salidas se añaden `ingestion_timestamp` y `source_file` para auditar cuándo y desde qué fichero se incorporó cada registro.

2.4.2. Capa silver: calidad, historial y entidades limpias

La capa silver se implementa en `codigo/src/medallion_pipeline/transformations/02_silver_transformation.py` con reglas centralizadas en `codigo/src/medallion_pipeline/rules/customers.py`. Su función es separar registros válidos y no válidos, normalizar tipos temporales y construir entidades coherentes para explotación analítica.

Tabla materializada	Filas	Contenido
<code>silver_customers_history</code>	3.050.000	Historial SCD2 del maestro de clientes mediante AUTO CDC. En Hito 2 hay una versión por cliente porque se parte de una única fotografía de contexto.
<code>silver_churn_events</code>	32.121.558	Eventos de uso limpios enriquecidos con la etiqueta mensual de churn. Mantiene una fila por cliente y mes, por eso conserva la cardinalidad de <code>bronze_usage</code> .
<code>silver_interactions_clean</code>	17.621.772	Interacciones válidas con <code>timestamp</code> convertido a tipo temporal y dominio de <code>interaction_type</code> validado.
<code>silver_quarantine_customers</code>	0	Registros de clientes que no superan las reglas de identificador, edad, contrato o tarifa.
<code>silver_quarantine_usage</code>	0	Registros de uso con identificador, periodo o métricas numéricas inválidas.
<code>silver_quarantine_labels</code>	0	Etiquetas sin cliente o sin periodo válido.
<code>silver_quarantine_interactions</code>	0	Interacciones sin cliente, sin timestamp o con tipo de interacción fuera del dominio controlado.

Columnas generadas en silver:

- `silver_customers_history`: columnas de cliente limpias más `__START_AT` y `__END_AT`.
- `silver_churn_events`: columnas de uso, `_rescued_data`, `ingestion_timestamp`, `source_file`, `usage_event_time`, `churn_date` y `label_available_date`.
- `silver_interactions_clean`: `agent_id`, `channel`, `customer_id`, `duration_seconds`, `interaction_type`, `resolution`, `satisfaction_score`, `timestamp`, `_rescued_data`, `ingestion_timestamp`, `source_file`.
- `silver_quarantine_customers`: columnas de `bronze_customers` más `customer_sequence_at`.

- `silver_quarantine_usage`: mismas columnas que `bronze_usage`.
- `silver_quarantine_labels`: `churn_date`, `customer_id`, `label_available_date`, `year_month`, `_rescued_data`, `ingestion_timestamp`, `source_file`.
- `silver_quarantine_interactions`: mismas columnas que `silver_interactions_clean`.

El uso de tablas de cuarentena evita descartar silenciosamente datos defectuosos y, al mismo tiempo, impide que registros inválidos contaminen la capa gold. Que las cuarentenas tengan cero filas en esta ejecución no significa que el mecanismo no exista; significa que el generador entregó datos compatibles con las reglas definidas para Hito 2.

2.4.3. Capa gold: base de modelado y características

La capa gold se reparte en tres scripts, igual que en el proyecto de referencia del profesor, donde la capa final se separa en spine, perfil y variables derivadas. En nuestro caso se mantiene la misma idea, pero adaptada al problema de churn telco:

- `03_gold_churn_spine.py` genera `gold_churn_spine`.
- `03_gold_customer_profile.py` genera `gold_customer_profile`.
- `03_gold_customer_aggregations.py` genera `gold_customer_aggregations`.

Tabla materializada	Filas	Contenido
<code>gold_churn_spine</code>	32.121.558	Tabla ancla para modelado supervisado. Contiene la clave cliente-mes, la fecha de disponibilidad de etiqueta, la variable objetivo binaria y señales de uso disponibles en el evento.
<code>gold_customer_profile</code>	3.050.000	Perfil estable del cliente con atributos demográficos, contractuales y dos derivadas simples para segmentación: grupo de edad y grupo de riesgo contractual.
<code>gold_customer_aggregations</code>	32.121.558	Features mensuales por cliente. Conserva métricas de uso, facturación, satisfacción y cobertura, y añade el ratio <code>bill_vs_data_ratio</code> .

Columnas generadas en gold:

- `gold_churn_spine`: `customer_id`, `year_month`, `usage_event_time`, `label_available_date`, `churn_date`, `label_will_churn`, `roaming_gb`, `sms_count`, `bill_overage`, `speed_mbps`, `days_active`.
- `gold_customer_profile`: `customer_id`, `age`, `gender`, `contract_type`, `region`, `region_type`, `tariff_plan`, `monthly_fee`, `num_lines`, `device_type`, `acquisition_channel`, `payment_method`, `signup_date`, `has_tv_bundle`, `has_fiber`, `has_roaming`, `paperless_billing`, `autopay`, `nps_score_at_start`, `is_active`, `__START_AT`, `__END_AT`, `age_group`, `contract_risk_group`.

- `gold_customer_aggregations`: `customer_id`, `year_month`, `window_end`, `label_available_date`, `label_will_churn`, `data_consumed_gb`, `call_minutes`, `bill_amount`, `days_payment_late`, `nps_score`, `coverage_score`, `bill_vs_data_ratio`.

Las tablas `gold_customer_profile` y `gold_customer_aggregations` se declaran con propiedades compatibles con Change Data Feed y claves temporales, de forma que puedan registrarse posteriormente como tablas de características en Unity Catalog y publicarse en Online Feature Store cuando el entorno académico permita Lakebase. En Hito 2 esta parte queda preparada a nivel de contrato de datos; el entrenamiento y la selección final de variables se abordan en Hito 3.

2.5. Control de calidad – Capa Silver

2.5.1. Centralización de reglas

Las reglas de calidad se centralizan en `codigo/src/medallion_pipeline/rules/customers.py` para evitar lógica duplicada y facilitar la revisión funcional. Esta centralización simplifica el mantenimiento: cuando una regla cambia, la actualización se realiza en un único punto y se propaga automáticamente al flujo completo. Cada regla es una expresión SQL que debe evaluar a `TRUE` para que un registro se considere válido; los registros que no la superan son marcados con el flag `is_quarantined` en lugar de descartados, preservando trazabilidad.

2.5.2. Reglas por entidad

Clientes (`get_customer_rules`) La entidad de clientes es el eje central del modelo de churn. Sus reglas buscan garantizar que cada registro represente a un cliente real e identificable, con datos de contrato y facturación coherentes.

Regla	Expresión SQL	Justificación
<code>valid_customer_id</code>	<code>customer_id IS NOT NULL</code>	Sin identificador no es posible vincular el registro con ninguna otra entidad del modelo. Es la clave de integridad referencial del sistema.
<code>valid_age</code>	<code>age >0 AND age <120</code>	Filtra edades biológicamente imposibles originadas por errores de ingesta o valores por defecto. El rango 1–119 cubre la totalidad de clientes reales posibles.

Regla	Expresión SQL	Justificación
valid_contract_type	contract_type IN ('monthly', 'annual')	El tipo de contrato determina la lógica de cálculo de churn. Valores fuera del dominio conocido indicarían un error de codificación upstream que podría distorsionar los modelos.
valid_monthly_fee	monthly_fee >= 0	Una tarifa negativa es económicamente incoherente y señala un error de transformación. Se permite el valor cero para cubrir planes gratuitos o promocionales.

Uso (get_usage_rules) Los registros de uso son la principal fuente de señal de comportamiento del cliente. La calidad en esta entidad es crítica porque sus valores alimentan directamente las features del modelo predictivo.

Regla	Expresión SQL	Justificación
valid_customer_id	customer_id IS NOT NULL	Necesario para asociar el consumo a un cliente concreto. Sin él el registro es inutilizable para el análisis.
valid_year_month	year_month IS NOT NULL	El período de referencia es imprescindible para la agregación mensual y la construcción de la ventana temporal del modelo.
valid_data_consumed	data_consumed_gb >= 0	El consumo de datos no puede ser negativo. Valores negativos apuntan a errores de medición o reversiones mal registradas.
valid_call_minutes	call_minutes >= 0	Análogamente, los minutos de llamada son una magnitud no negativa. Un valor negativo indicaría un error en el sistema de medición.
valid_bill_amount	bill_amount >= 0	El importe facturado debe ser cero o positivo. Importes negativos pueden deberse a abonos incorrectamente clasificados que contaminarían los ratios de facturación.

Etiquetas (`get_label_rules`) Las etiquetas representan la variable objetivo del modelo de churn. Su integridad es especialmente sensible: un error aquí no degrada la calidad del dato sino directamente la del entrenamiento.

Regla	Expresión SQL	Justificación
<code>valid_customer_id</code>	<code>customer_id IS NOT NULL</code>	Una etiqueta sin cliente asociado no puede incorporarse al dataset de entrenamiento ni de evaluación.
<code>valid_year_month</code>	<code>year_month IS NOT NULL</code>	La etiqueta de churn es inherentemente temporal. Sin período de referencia no es posible alinearla con las features del mes correspondiente.

Interacciones (`get_interaction_rules`) Los eventos de interacción enriquecen el perfil del cliente con señales de comportamiento cualitativo. La validación se centra en la coherencia del tipo de interacción, ya que determina cómo se agregan y ponderan los eventos en la capa gold.

Regla	Expresión SQL	Justificación
<code>valid_customer_id</code>	<code>customer_id IS NOT NULL</code>	Sin cliente identificado la interacción no puede vincularse al historial correspondiente.
<code>valid_timestamp</code>	<code>timestamp IS NOT NULL</code>	La ordenación temporal de las interacciones es necesaria para construir features de recencia y frecuencia. Sin timestamp el evento pierde su dimensión temporal.
<code>valid_interaction_type</code>	<code>interaction_type IN (...)</code>	El tipo de interacción está codificado en un dominio cerrado de 13 valores. Cualquier valor fuera de este dominio indica un error de integración upstream y podría introducir categorías espurias en el modelo.

2.6. Robustez y resolución de incidencias

La robustez del Hito 2 no se plantea como una lista de errores internos de desarrollo, sino como un conjunto de decisiones concretas de ingeniería y operación que hacen reproducible el pipeline:

- Separación estricta entre código versionable y datos generados. Los directorios `context/`, `events/`, `source_buffer/` y los logs del generador se excluyen del bundle para que `validate` y `deploy` no intenten sincronizar artefactos masivos.
- Imports robustos para ejecución en Databricks. Las reglas de calidad viven en `src/medallion_pipeline/rules/` y se cargan desde el `bundle.sourcePath` inyectado por el pipeline, manteniendo la separación entre reglas y transformaciones.
- Capa silver con cuarentena explícita. Cada entidad tiene un flujo de evaluación y una tabla de cuarentena, lo que permite que el pipeline siga siendo trazable aunque en futuras cargas aparezcan registros inválidos.
- Enriquecimiento de eventos con patrón stream-static. Para Hito 2 se mantiene `bronze_usage` como flujo incremental y se cruza con una vista estática de etiquetas mensuales válidas.
- Ejecución en modo triggered. El pipeline no queda ejecutándose de forma continua; se lanza bajo demanda o desde el job, procesa el lote disponible y libera recursos.

Con estos ajustes, la evidencia relevante es el resultado final: actualización completa en estado `COMPLETED`, tablas materializadas por capa y contrato de datos preparado para modelado.

2.7. Orquestación final del hito

Con la capa de datos estabilizada, cerramos el hito incorporando un job final de orquestación, equivalente en estructura al del proyecto de referencia:

- Job: Telco Churn - Hito 2 Orchestration
- Secuencia:
 1. Ejecución de pipeline Medallion,
 2. Ejecución del notebook `codigo/notebooks/04_Feature_Store_Registration.py`.

El notebook final se incorporó en `codigo/notebooks` y se adaptó para ejecución en modo simulación de Feature Store, ya que Free Edition no permite desplegar Lakebase/Online Feature Store. Aun así, el notebook valida la existencia de tablas gold, registra trazas de ejecución y conserva comentado el bloque de código real para activar en cuanto haya licencia.

También se configuraron notificaciones de éxito y error en el propio job para ambos integrantes del equipo, de forma que la ejecución quede monitorizada sin necesidad de revisar manualmente la UI en cada corrida.

2.8. Decisiones de limpieza y gobierno del entorno

Como parte del cierre del hito se realizó limpieza de recursos antiguos para evitar duplicidades y confusión operativa. Se eliminaron volumen y objetos del esquema

previo ya en desuso, manteniendo únicamente los recursos oficiales del proyecto en `workspace.telco_churn`.

Esta parte, aunque no cambia el resultado analítico, sí mejora la mantenibilidad del entorno: reduce riesgo de ejecutar sobre tablas antiguas y simplifica la explicación del proyecto durante defensa.

2.9. Estado de cierre del Hito 2

El cierre técnico queda validado con:

- Pipeline en estado **COMPLETED** (update final validada: `aae8711c-6d5f-4502-b237-41f6b738c0e6`),
- Job de orquestación ejecutado en **SUCCESS** (`run_id: 640253146512278`),
- Tablas medallion materializadas en `workspace.telco_churn`,
- Recursos antiguos del esquema previo limpiados para evitar ambigüedad operativa.

El DAG resultante refleja la linaje completa del pipeline, desde la ingesta en bronce hasta las tablas gold, con todas las tablas en estado **COMPLETED** y los flujos de cuarentena activos por entidad:



Figura 1: DAG del pipeline medallion — Hito 2

En términos académicos y técnicos, el objetivo del hito se considera cumplido: no solo existe código, sino una solución desplegada, ejecutada y trazable, con documentación reproducible tanto por CLI como por UI. Por tanto, Hito 2 queda cerrado con una base de datos estable y gobernada para iniciar Hito 3 sin rehacer ingeniería de datos.

3. Hito 3: Modelado y experimentación

3.1. Objetivo del hito

El tercer hito convierte la base de datos gobernada del Hito 2 en un ciclo de modelado reproducible para predicción de churn. El objetivo no es únicamente entrenar un clasificador, sino dejar trazado el proceso completo: generación del dataset de entrenamiento con garantías temporales, experimentación con MLflow, registro del mejor candidato en Unity Catalog y evaluación champion-challenger antes de aceptar o rechazar un modelo.

La guía oficial separa esta fase del pipeline declarativo Medallion. Por tanto, el código de datos sigue viviendo en `src/medallion_pipeline/transformations/` y el modelado se implementa en `notebooks/`, orquestado mediante un Job de Databricks. No se han añadido notebooks de ML a `resources/telco_churn.pipeline.yml` porque ese recurso define el pipeline de datos; forzar ahí el entrenamiento rompería la separación recomendada entre ingeniería de datos y MLOps.

3.2. Componentes implementados

La fase queda distribuida en notebooks y utilidades con responsabilidades separadas:

- `05_Training_Dataset_Generation.ipynb`: materializa `gold_churn_training_dataset` desde las tablas `gold` mediante Feature Engineering y `FeatureLookup`.
- `07_Utils.py`: centraliza configuración, partición temporal, columnas, pipeline Spark MLlib, métricas y funciones de evaluación.
- `07_Training_Job.ipynb`: entrena una configuración concreta de regresión logística en una sesión aislada.
- `07_Evaluation_Job.ipynb`: evalúa un modelo serializado sobre el split indicado y devuelve métricas.
- `07_MLflow_Experimentation.ipynb`: ejecuta el grid search, registra artefactos y promueve el mejor modelo al alias `candidate`.
- `08_Utils.py`: encapsula la lógica champion-challenger, gestión de alias y registro de decisiones.
- `08_Production.ipynb`: evalúa el candidato en test, compara contra `champion` y decide promoción o rechazo.

La orquestación reproducible se define en `resources/telco_churn_ml.job.yml` mediante el job `telco_churn_ml_orchestration`, con tres tareas secuenciales: `run_training_dataset_gen`, `run_mlflow_experimentation` y `run_production`.

3.3. Generación del dataset de entrenamiento

El dataset de entrenamiento se construye desde la tabla `workspace.telco_churn.gold_churn_s` y se enriquece con las tablas `gold_customer_profile` y `gold_customer_aggregations`.

La unión usa claves temporales para que las características correspondan al estado disponible en el momento de predicción (`usage_event_time`), evitando fuga de información futura.

Las columnas `churn_date` y `label_available_date` se excluyen del dataset materializado porque no estarían disponibles como señales reales de inferencia. La tabla resultante se guarda como Delta estática en `workspace.telco_churn.gold_churn_training_dataset`, con propiedades de trazabilidad:

Propiedad	Valor validado
<code>ml.delta_semantic_version</code>	0
<code>ml.delta_physical_version</code>	12
<code>ml.data_max_date</code>	2024-12-01
<code>ml.data_previous_max_date</code>	vacío en el primer ciclo
<code>ml.num_rows</code>	16.316.445
<code>ml.num_features</code>	33

La comprobación de calidad confirma que el dataset contiene 16.316.445 filas, 2.042.162 clientes distintos y ventana temporal de 2023-07-01 a 2024-12-01. No se detectaron nulos en las columnas materializadas. El balance global es de 788.672 filas con churn (4,83 %) y 15.527.773 sin churn (95,17 %), por lo que el problema queda desbalanceado y justifica el uso de pesos inversos de frecuencia en el clasificador.

La partición temporal validada fue:

Split	Periodo	Filas	Churn	No churn	% churn
Entrenamiento	2023-07-01 a 2024-06-30	9.900.403	508.454	9.391.949	5,14 %
Validación	2024-07-01 a 2024-09-30	3.262.300	144.869	3.117.431	4,44 %
Prueba	2024-10-01 a 2024-12-01	3.153.742	135.349	3.018.393	4,29 %

3.4. Arquitectura del pipeline de modelado

El modelo se empaqueta como un único **Pipeline** de Spark MLlib para que el pre-procesado aprendido durante entrenamiento se aplique de forma idéntica en validación, prueba y producción. Esta decisión evita divergencias entre entrenamiento e inferencia.

Las etapas principales son:

1. Imputación con mediana para variables numéricas.
2. Conversión de indicadores booleanos/binarios.
3. Ingeniería de variables derivadas disponibles en inferencia, como `tenure_months`, `is_high_bill`, `is_late_payer`, `low_nps_flag` y `bill_vs_data_ratio`.

4. Indexación de categóricas con `StringIndexer`.
5. Codificación `OneHotEncoder`.
6. Ensamblado en vector de características.
7. Selección por varianza con `VarianceThresholdSelector`.
8. Escalado estándar.
9. Clasificación con `LogisticRegression`.

Se eligió regresión logística porque ofrece una primera línea base robusta, interpretable y razonable para el volumen de datos disponible. En un proyecto de retención, la explicabilidad del modelo es relevante: no basta con maximizar métricas, también conviene entender qué señales elevan el riesgo de fuga para poder diseñar acciones comerciales.

3.5. Experimentación con MLflow

La experimentación se ejecutó como búsqueda completa sobre ocho configuraciones:

<code>reg_param</code>	<code>elastic_net</code>	<code>max_iter</code>	AUC-PR val.	AUC-ROC val.	F1 val.	Umbral
0.01	0.0	100	0.988422	0.999053	0.993917	0.68
0.01	0.0	200	0.988422	0.999053	0.993917	0.68
0.01	0.5	100	0.990393	0.999233	0.995061	0.64
0.01	0.5	200	0.990393	0.999233	0.995061	0.64
0.1	0.0	100	0.966975	0.996650	0.990864	0.61
0.1	0.0	200	0.966975	0.996650	0.990864	0.61
0.1	0.5	100	0.964275	0.994287	0.992659	0.55
0.1	0.5	200	0.964274	0.994287	0.992659	0.55

La métrica primaria fue AUC-PR, más adecuada que accuracy o AUC-ROC cuando la clase positiva es minoritaria. El mejor modelo fue la configuración `reg_param = 0.01`, `elastic_net_param = 0.5` y `max_iter = 100`, con AUC-PR de validación 0.990393, AUC-ROC 0.999233, F1 0.995061, precisión 0.995162, recall 0.995007 y umbral óptimo 0.64.

El experimento quedó registrado en MLflow en:

- Experimento: `/Users/alonso.marcos@alu.uclm.es/telco_churn_churn_detection_training`
- Parent run: `5cab64c09efe458e904e625678fa1cca`
- Mejor run: `ec2ec49b626444f58f3b2db04ed44ee7`
- Modelo registrado: `workspace.telco_churn.churn_lr_pipeline`

3.6. Registro en Unity Catalog y ciclo champion-challenger

El modelo se gobierna con alias de Unity Catalog. En el primer ciclo validado, la versión 2 quedó como **champion** tras el arranque en frío. En la ejecución completa posterior del job, el nuevo candidato se registró como versión 3, se evaluó contra el champion y quedó marcado como **rejected** porque no mejoró estrictamente la métrica primaria en test.

El estado final validado es:

Alias	Versión	Significado
champion	2	Modelo actualmente aceptado para producción.
rejected	3	Candidato evaluado que no mejora al champion.

La evaluación de producción del run 499bacbcd647f8a6342901b3419888 produjo:

Modelo	AUC-PR test	AUC-ROC test	F1 test	Decisión
Champion v2	0.990086	0.999231	0.996431	Se mantiene
Challenger v3	0.990086	0.999231	0.996431	Se rechaza por no mejorar estrictamente

La decisión es conservadora y coherente con un ciclo de producción: si un challenger empatiza al champion, no aporta valor operativo suficiente para cambiar el modelo aceptado. La tabla baseline para monitorización quedó materializada como `workspace.telco_churn.gold_churn_test` con 3.153.742 filas y ventana 2024-10-01 a 2024-12-01.

3.7. Orquestación y evidencia de ejecución

El job completo de Hito 3 se desplegó mediante Databricks Asset Bundles y quedó disponible en Jobs & Pipelines como:

- Job: [dev alonso_marcos] Telco Churn - Hito 3 ML Orchestration
- Job ID: 588950994995073
- Run validado: 329240873651157
- Estado final: SUCCESS

Las tareas finalizaron así:

Tarea	Estado	Duración aprox.
run_training_dataset_generation	SUCCESS	229 s
run_mlflow_experimentation	SUCCESS	7.390 s
run_production	SUCCESS	770 s

También se ejecutó antes una prueba parcial de producción (`run_id: 817389537042464`) para aislar el fallo reportado por Jose. Esa prueba permitió confirmar que el problema no era realmente que el modelo fuera demasiado grande, sino una combinación de registro MLflow en el sitio incorrecto, alias `candidate` no idempotente tras reintentos y ruta de experimento con carpeta padre inexistente. Las correcciones aplicadas fueron:

- Mover el registro del modelo al notebook orquestador de MLflow, donde existe el `run_id` correcto.
- Hacer que producción pueda continuar si un reintento encuentra el alias `challenger` en vez de `candidate`.
- Usar un path de experimento directamente bajo `/Users/<usuario>/...`
- Corregir la tabla baseline de `gold_fraud_test_baseline` a `gold_churn_test_baseline`.
- Limpiar textos heredados del ejemplo de fraude para mantener coherencia con churn.

3.8. Cierre del Hito 3

El Hito 3 queda cerrado con un ciclo de modelado reproducible y gobernado: existe dataset de entrenamiento versionado, búsqueda de hiperparámetros con MLflow, modelo registrado en Unity Catalog, alias de producción coherentes, baseline de test y job end-to-end validado. El pipeline Medallion del Hito 2 permanece separado y no se modifica para ejecutar ML, porque la guía reserva esa fase a notebooks/jobs independientes.

4. Hito 4: Despliegue y monitorización (apartado reservado)

Sección reservada para completar en la última entrega.